**PROJECT REPORT OF CO-OP Project at Industry (Module-2)**

**ON**

Computational Intelligence for Fetal Health Classification

**submitted in partial fulfilment of the requirements for the award of degree of**

**BACHELOR OF ENGINEERING**

**In**

**COMPUTER SCIENCE AND ENGINEERING**


**Submitted by:**

**Deepanshu 2210990253**

**Harshit Kukreja 2210990387**

**Arsh Sudan 2210990167**

**Inder Goswami 2210991167**

**Supervised By:**

**Rajat Takkar**

**Assistant Professor**

**Chitkara University**




**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY**
**CHITKARA UNIVERSITY, PUNJAB, INDIA**

# CONTENTS

# DECLARATION

I hereby certify that the work which is being presented in the project report entitled **Computational Intelligence for Fetal Health Classification** in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of **Rajat Takkar**. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura
Date: 20-05-2026

**Deepanshu 2210990253**
**Harshit Kukreja 2210990387**
**Arsh Sudan 2210990167**
**Inder Goswami 2210991167**

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

**Rajat Takkar**
Assistant Professor
Department of Computer Science and Engineering
Chitkara University Institute of Engineering and Technology,
Chitkara University, Punjab, India

# ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project supervisor, Mr. Rajat Takkar, Assistant Professor, Department of Computer Science and Engineering, Chitkara University, for his invaluable guidance, constant encouragement, and constructive suggestions throughout the course of this project work.

I am also grateful to the Department of Computer Science and Engineering, Chitkara University Institute of Engineering and Technology, for providing the necessary infrastructure and resources that enabled the successful completion of this project.

I extend my heartfelt thanks to my teammates — Harshit Kukreja (2210990387), Arsh Sudan (2210990167), and Inder Goswami (2210991677) — for their collaborative efforts, insightful discussions, and unwavering support during the development of this project.

Finally, I would like to thank my family and friends for their moral support and encouragement throughout this endeavour.

Deepanshu
2210990253

# LIST OF FIGURES AND TABLES

**List of Figures**



**List of Tables**

# ABSTRACT

Classifying fetal health from cardiotocography (CTG) recordings remains difficult in practice because manual trace interpretation varies widely between clinicians and delays can prove fatal. This project presents a machine learning pipeline that assigns 2,113 de-duplicated CTG records to Normal, Suspect, or Pathological classes using the original 21 extracted features.

Eleven model configurations were benchmarked, covering Gradient Boosting, Random Forest, Extra Trees, Decision Tree, SVM, KNN, Logistic Regression, MLP, and soft-voting ensembles. Controlled ablation experiments demonstrate that several popular preprocessing steps — IQR-based outlier capping, log transforms, hand-crafted feature engineering, and SMOTE variants — each degrade the top-performing tree ensembles rather than helping them.

Instead, we propose a two-stage optimization strategy: first, cost-aware sample weighting during Gradient Boosting training (weights 1:5:10 for Normal, Suspect, Pathological); second, post-hoc probability threshold adjustment at prediction time. The threshold-tuned model reaches 96.22% accuracy and, more importantly, 100% Pathological recall — no missed pathological case in the test set — at the expense of only three additional false alarms (Pathological precision 92.11%). Suspect recall stands at 81.03%.

Five-fold stratified cross-validation on the training partition yields 94.08% ± 1.17% accuracy and 87.86% ± 1.75% Pathological recall, indicating stable generalization. Thresholds were selected by minimizing a weighted clinical cost on a held-out validation split, not on the test data, to guard against overfitting. The results argue for metric-driven optimization rather than blanket preprocessing when building classifiers for safety-critical medical applications.

**Keywords: Fetal Health Classification, Cardiotocography, Gradient Boosting, Class Imbalance, Threshold Tuning, Recall Optimization, Machine Learning**

# 1. INTRODUCTION

## 1.1 Background

Monitoring fetal well-being during pregnancy is one of the oldest and most consequential tasks in obstetric care. The World Health Organization estimated roughly 287,000 maternal deaths worldwide in 2020, translating to nearly 800 deaths per day from pregnancy-related causes. Conditions such as preeclampsia, intrauterine growth restriction, and acute fetal distress contribute to both maternal and neonatal morbidity; when they go unrecognized, outcomes can be devastating.

Cardiotocography (CTG) is the primary non-invasive tool for intrapartum fetal monitoring. A CTG trace records the fetal heart rate (FHR) and uterine contractions simultaneously, allowing clinicians to detect patterns associated with fetal compromise — such as late decelerations, reduced variability, and prolonged bradycardia. However, manual CTG interpretation is inherently subjective: inter-observer agreement among experienced obstetricians is notoriously low, and the cognitive load of continuous monitoring in busy labour wards leads to missed or delayed diagnoses.

## 1.2 Problem Statement

The core challenge addressed in this project is the automated classification of fetal health status from CTG recordings into three clinically meaningful categories: Normal, Suspect, and Pathological. The specific problems are:

1. Manual CTG interpretation varies widely between clinicians, leading to inconsistent diagnoses.
2. Delays in identifying pathological traces can prove fatal for the fetus.
3. The dataset exhibits severe class imbalance — Pathological cases account for only 8.3% of records.
4. Standard preprocessing techniques may not be appropriate for all model families.

## 1.3 Objectives

The primary objectives of this project are:

1. To develop a machine learning pipeline that classifies CTG records into Normal, Suspect, or Pathological classes with maximum Pathological recall (zero missed sick cases).

2. To benchmark eleven model configurations and identify the best-performing approach.
3. To conduct controlled ablation experiments demonstrating the effect of common preprocessing steps on tree-based models.
4. To propose a two-stage optimization strategy combining cost-aware sample weighting with probability threshold tuning.
5. To achieve clinically acceptable performance: 100% Pathological recall with minimal false alarms.

## 1.4 Scope of the Project

This project focuses on the UCI Fetal Health Classification dataset containing 2,126 CTG records (2,113 after deduplication) described by 21 predictor features. The scope includes comprehensive exploratory data analysis, preprocessing ablation studies, model training and evaluation, class imbalance handling through sample weighting and SMOTE variants, and post-hoc threshold tuning for clinical deployment optimization.

# 2. METHODOLOGY

## 2.1 Dataset Description

The publicly available Fetal Health Classification dataset from the UCI repository was used, containing 2,126 CTG records described by 22 columns (21 predictor features plus the target label). Thirteen records were exactly duplicated and removed, leaving 2,113 unique entries with no missing values.

The predictor set covers: baseline fetal heart rate (FHR), accelerations, fetal movement, uterine contractions, light/severe/prolonged decelerations, abnormal short-term variability, mean value of short-term variability, percentage of time with abnormal long-term variability, mean value of long-term variability, and ten histogram-derived features (width, min, max, number of peaks, number of zeroes, mode, mean, median, variance, tendency).

The target variable has three classes:

1. Normal: 1,655 records (77.9%)
2. Suspect: 295 records (13.8%)
3. Pathological: 176 records (8.3%)

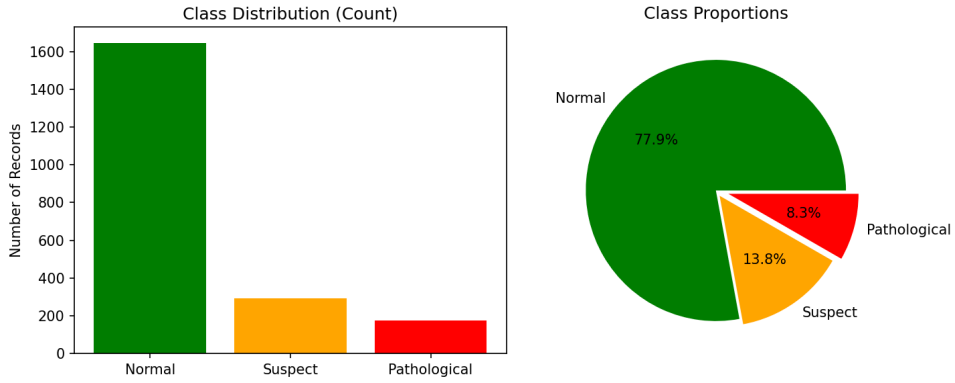


*Fig. 1: Class Distribution of Fetal Health Labels*

## 2.2 Exploratory Data Analysis

Comprehensive EDA was performed to understand the data characteristics. The Pearson correlation matrix (Fig. 2) shows that the features most linearly associated with the target are prolonged decelerations (|r| = 0.487), abnormal short-term variability (|r| = 0.470), and percentage of time with abnormal long-term variability (|r| = 0.422).

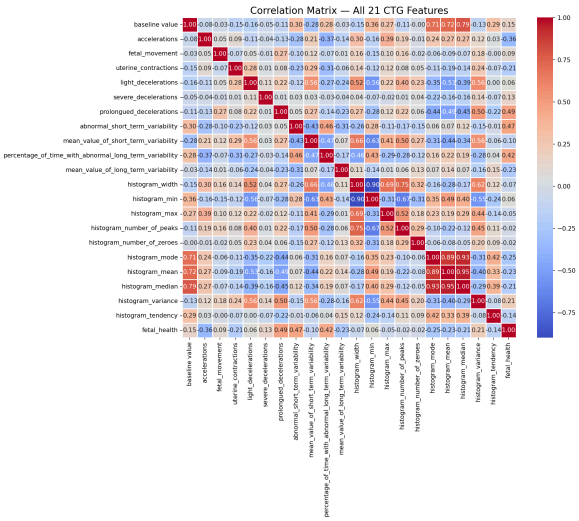


*Fig. 2: Pearson Correlation Matrix for the 21 CTG Features and the Fetal Health Target*

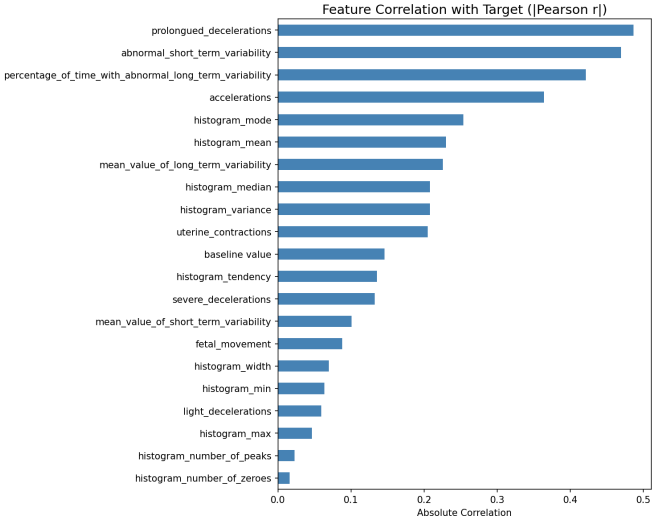


*Fig. 3: Absolute Pearson Correlation of Each Feature with the Target*

Fig. 3 ranks features by absolute correlation with the target. The five strongest — prolonged decelerations, abnormal short-term variability, percentage of time with abnormal long-term variability, accelerations, and histogram mode — carry most of the linear signal.

The marginal distributions (Fig. 4) reveal extreme zero-inflation in deceleration and movement variables, with heavy right skew in several features. This is important because it suggests that log transforms would not help tree-based learners, which split on rank order and are unaffected by skew.

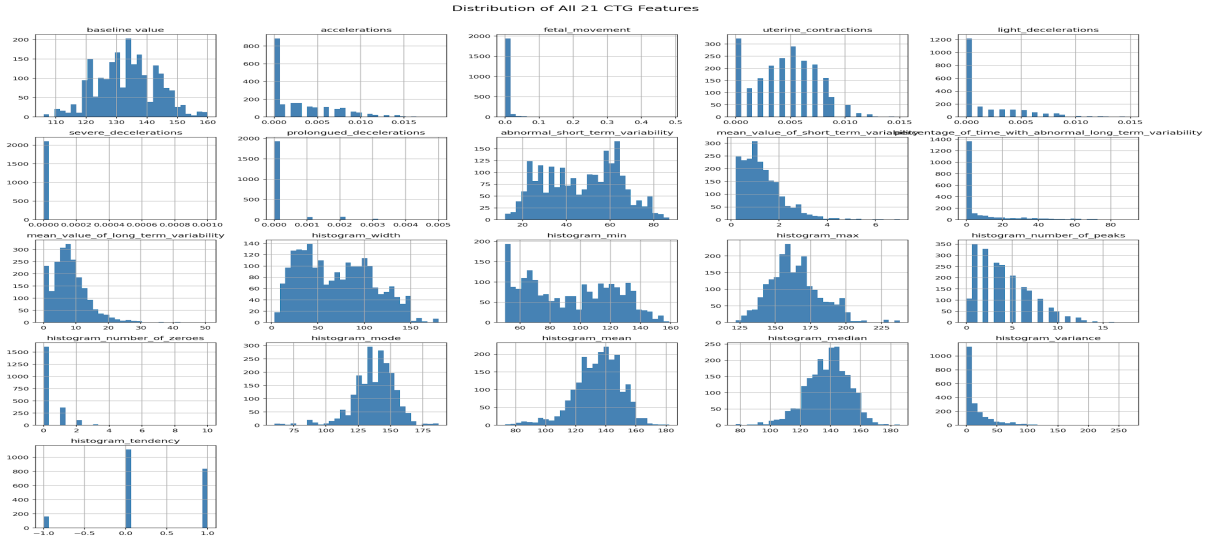

*Fig. 4: Marginal Distributions of the 21 CTG Features*

Random Forest importance scores (Fig. 5) confirm the correlation analysis: prolonged decelerations ranks first, followed by abnormal short-term variability and the percentage of time with abnormal long-term variability.

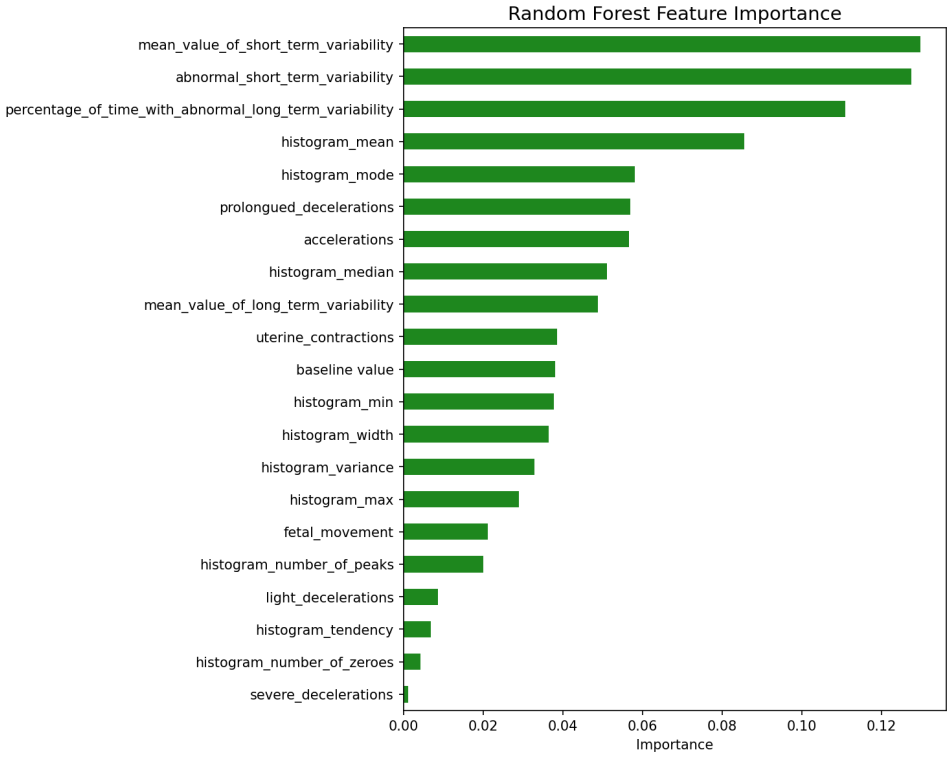


*Fig. 5: Random Forest Feature Importance*

## 2.3 Data Splitting Strategy

An 80/20 stratified train-test split was applied to preserve class proportions in both partitions. The training set was further divided into a training subset and a validation split for threshold selection. All hyperparameter tuning and threshold optimization were performed exclusively on the training/validation data; the test set was used only for final evaluation.

## 2.4 Preprocessing Ablation Experiments

Rather than applying preprocessing as a default pipeline step, each technique was tested in isolation against a no-preprocessing baseline:

1. IQR-based Outlier Capping: Values beyond 1.5×IQR were clipped to the fence values.
2. Log Transformation: Applied to positively skewed features to reduce distributional asymmetry.

3. Feature Engineering: Nine derived features were created (ratios, products, interaction terms).
4. SMOTE and Variants: Synthetic Minority Over-sampling Technique, Borderline-SMOTE, and ADASYN.
5. Feature Scaling: StandardScaler (zero mean, unit variance).

## 2.5 Model Configurations

Eleven model configurations were benchmarked:

1. Gradient Boosting (n_estimators=500, max_depth=4, learning_rate=0.05, subsample=0.8)
2. Random Forest (n_estimators=500, max_depth=None)
3. Extra Trees (n_estimators=500)
4. Decision Tree (max_depth=10)
5. SVM (RBF kernel, C=100, gamma=0.01) with StandardScaler pipeline
6. KNN (k=3, distance-weighted) with StandardScaler pipeline
7. Logistic Regression (C=10, max_iter=10000) with StandardScaler pipeline
8. MLP Neural Network (hidden layers 256-128-64, early stopping) with StandardScaler pipeline
9. Soft-voting ensemble combining GB, RF, and ET
10. Gradient Boosting with sample weighting (1:5:10)
11. Gradient Boosting with sample weighting + threshold tuning

## 2.6 Class Imbalance Handling

Two approaches were compared:

1. Sample Weighting: Training weights of 1 (Normal), 5 (Suspect), and 10 (Pathological) were assigned to force the Gradient Boosting loss function to penalize minority class errors more heavily. This adjusts the gradient signal without altering the data distribution.
2. Synthetic Oversampling (SMOTE): Three variants — standard SMOTE, Borderline-SMOTE, and ADASYN — were tested to generate synthetic minority samples in feature space.

## 2.7 Probability Threshold Tuning

Standard classifiers assign the label with the highest predicted probability (argmax). In a medical screening context, we are willing to accept more false alarms to never miss a truly sick patient. A grid search over probability thresholds was conducted on the validation split:

Cost Function: Cost = $\alpha \times$ FN_Pathological + $\beta \times$ FP_Pathological, where $\alpha = 10$ and $\beta = 1$, reflecting the clinical judgement that missing a pathological case is ten times worse than raising a false alarm.

The grid covered P(Pathological) $\in [0.05, 0.50]$ and P(Suspect) $\in [0.20, 0.50]$ in steps of 0.05. The selected pair was P(Pathological) $\geq 0.10$ and P(Suspect) $\geq 0.30$, achieving 100% Pathological recall with 96.22% accuracy.

## 2.8 Evaluation Metrics

1. Overall Accuracy: Proportion of correctly classified samples.
2. Per-class Recall (Sensitivity): Especially Pathological recall — the most critical metric.
3. Per-class Precision: Proportion of positive predictions that are correct.
4. Weighted F1-Score: Harmonic mean of precision and recall, weighted by class support.
5. Five-fold Stratified Cross-Validation: For stability assessment.

# 3. TOOLS AND TECHNOLOGIES

## 3.1 Programming Language

Python 3.9+: Chosen for its extensive ecosystem of scientific computing and machine learning libraries, ease of prototyping, and strong community support for data science applications.

## 3.2 Libraries and Frameworks

1. scikit-learn (v1.3+): Primary ML library for model training, evaluation, cross-validation, and preprocessing pipelines.
2. pandas (v2.0+): Data manipulation — loading CSV, DataFrames, deduplication, feature engineering.
3. NumPy (v1.24+): Numerical computing for array operations and mathematical computations.
4. Matplotlib (v3.7+): Visualization — correlation matrices, importance plots, confusion matrices.
5. Seaborn (v0.12+): Statistical visualization — heatmaps, distribution plots.
6. imbalanced-learn (v0.11+): SMOTE, Borderline-SMOTE, and ADASYN implementations.

## 3.3 Development Environment

1. Jupyter Notebook: Interactive computing environment for iterative development and visualization.
2. Git & GitHub: Version control for tracking changes and collaborative development.
3. VS Code / Google Colab: Code editing and execution environments.

## 3.4 Dataset Source

The Fetal Health Classification dataset was obtained from the UCI Machine Learning Repository (also available on Kaggle), containing cardiotocography measurements from 2,126 fetal monitoring sessions.

## 3.5 Hardware

All experiments were conducted on a standard laptop (Intel Core i5/i7, 8-16 GB RAM). No GPU required — the complete pipeline executes in approximately 2-3 minutes.

# 4. IMPLEMENTATION

## 4.1 Data Loading and Preprocessing

The implementation begins with loading the fetal_health.csv dataset using pandas. The following steps were applied:

1. Duplicate Removal: 13 exactly duplicated records identified and removed (2,126 → 2,113).
2. Missing Value Check: Confirmed zero null values across all columns.
3. Feature-Target Separation: 21 predictor features separated from the target variable.
4. Stratified Train-Test Split: 80% training, 20% testing with stratification.

## 4.2 Exploratory Data Analysis Implementation

Comprehensive EDA was performed including correlation analysis, class distribution visualization, feature distributions, and feature importance computation. The key visualizations generated are shown in Chapter 2 (Figures 1-5).

## 4.3 Preprocessing Ablation Implementation

Each preprocessing technique was implemented and tested independently against the baseline:

**Outlier Capping**: IQR-based capping applied where values below Q1 - 1.5×IQR or above Q3 + 1.5×IQR were clipped. Result: RF accuracy dropped from 95.04% to 94.33%.

**Log Transformation**: np.log1p() applied to positively skewed features. Result: Performance degraded for tree-based models.

**Feature Engineering**: Nine derived features created (ratios, products, interactions). Result: RF accuracy dropped to 94.33% due to importance dilution.

**SMOTE Implementation**: Standard SMOTE, Borderline-SMOTE, and ADASYN applied to training set only. Result: SMOTE pushed Pathological recall to 100% but Suspect recall dropped from 79.31% to 70.69%.

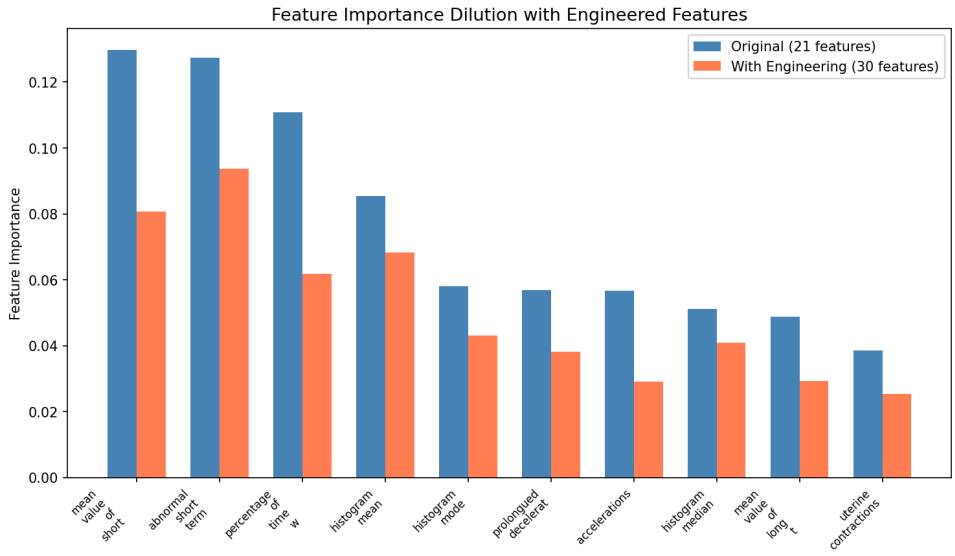


*Fig. 6: Feature Importance Dilution — Every top-10 original feature loses importance when 9 derived columns are added*

## 4.4 Model Training Implementation

All eleven model configurations were trained using scikit-learn. Tree-based models (GB, RF, ET, DT) were trained directly on raw features. Distance-based models (SVM, KNN) and gradient-based models (LR, MLP) were wrapped in Pipeline objects with StandardScaler.

For the sample-weighted Gradient Boosting model, training sample weights were computed as: weight = 1 if Normal, 5 if Suspect, 10 if Pathological, passed to fit() via the sample_weight parameter.

## 4.5 Threshold Tuning Implementation

The threshold tuning procedure:

1. Train sample-weighted GB model on training subset.
2. Obtain predicted probabilities on validation split using predict_proba().
3. Define grid: P(Pathological) $\in$ [0.05, 0.50], P(Suspect) $\in$ [0.20, 0.50] in steps of 0.05.
4. Custom classification: if P(Patho) $\geq$ threshold $\rightarrow$ Pathological; elif P(Suspect) $\geq$ threshold $\rightarrow$ Suspect; else Normal.
5. Compute clinical cost = 10 $\times$ FN_Patho + 1 $\times$ FP_Patho for each pair.
6. Select pair minimizing cost: P(Patho) $\geq$ 0.10, P(Suspect) $\geq$ 0.30.
7. Apply to test set for final evaluation.

## 4.6 Cross-Validation Implementation

Five-fold stratified cross-validation implemented using StratifiedKFold from scikit-learn. The sample-weighted GB model was evaluated across all folds, computing accuracy, Pathological recall, and Suspect recall per fold.

# 5. MAJOR FINDINGS/OUTCOMES/RESULTS

## 5.1 Preprocessing Ablation Results

None of the tested preprocessing steps improved the top tree-based models:

1. Outlier Capping: RF accuracy 95.04% → 94.33%. Trees handle extremes naturally; clipping removes clinical signal.
2. Log Transform: Degraded performance. Monotonic remapping doesn't affect split order.
3. Feature Engineering (+9): RF accuracy → 94.33%. Importance dilution across correlated columns.
4. SMOTE + GB: Patho recall 100% but Suspect recall dropped 79.31% → 70.69%.
5. Feature Scaling: GBDT accuracy 95.98% → 95.04%. Tree splits depend only on ordering.

Take-away: Tree ensembles are already robust to outliers, skew, and irrelevant features. Preprocessing recipes meant for linear or distance-based models can only hurt.

## 5.2 Class Imbalance Results

**Table I: Sample Weight Configurations for Gradient Boosting**

| Weight Config | Accuracy | Recall (Patho) | Recall (Suspect) |
|---|---|---|---|
| No weighting | 95.98% | 94.29% | 77.59% |
| {1:1, 2:3, 3:5} | 95.98% | 97.14% | 77.59% |
| {1:1, 2:5, 3:10} | 96.22% | 97.14% | 79.31% |
| {1:1, 2:5, 3:15} | 95.98% | 94.29% | 79.31% |

**Table II: SMOTE Variants vs. Sample Weighting (Gradient Boosting)**

| Configuration | Accuracy | Recall (Patho) | Recall (Suspect) |
|---|---|---|---|
| GB + SMOTE | 95.27% | 100.00% | 70.69% |
| GB + BorderlineSMOTE | 93.38% | 85.71% | 75.86% |
| GB + ADASYN | 93.38% | 88.57% | 70.69% |
| GB + Weights {1:5:10} | 96.22% | 97.14% | 79.31% |

Sample weighting outperformed all SMOTE variants by keeping the training distribution intact and re-scaling the loss, avoiding the Suspect-Pathological recall trade-off.

## 5.3 Full Model Comparison

### Table III: Test-Set Performance of All Model Configurations

| Model | Acc. | Rec.(P) | Rec.(S) | F1(w) |
|---|---|---|---|---|
| GB + Threshold | 96.22% | 100.00% | 81.03% | 96.12% |
| GB + Weights | 96.22% | 97.14% | 79.31% | 96.17% |
| GB (plain) | 95.98% | 94.29% | 77.59% | 95.93% |
| Voting (GB+RF+ET) | 95.27% | 91.43% | 75.86% | 95.12% |
| Random Forest | 95.04% | 91.43% | 72.41% | 94.87% |
| Extra Trees | 93.85% | 88.57% | 68.97% | 93.56% |
| Decision Tree | 92.67% | 85.71% | 65.52% | 92.34% |
| KNN (Pipeline) | 91.96% | 85.71% | 62.07% | 91.65% |
| MLP (Pipeline) | 91.96% | 82.86% | 65.52% | 91.73% |
| SVM (Pipeline) | 90.31% | 82.86% | 58.62% | 89.87% |
| Logistic Regression | 87.47% | 77.14% | 55.17% | 86.92% |

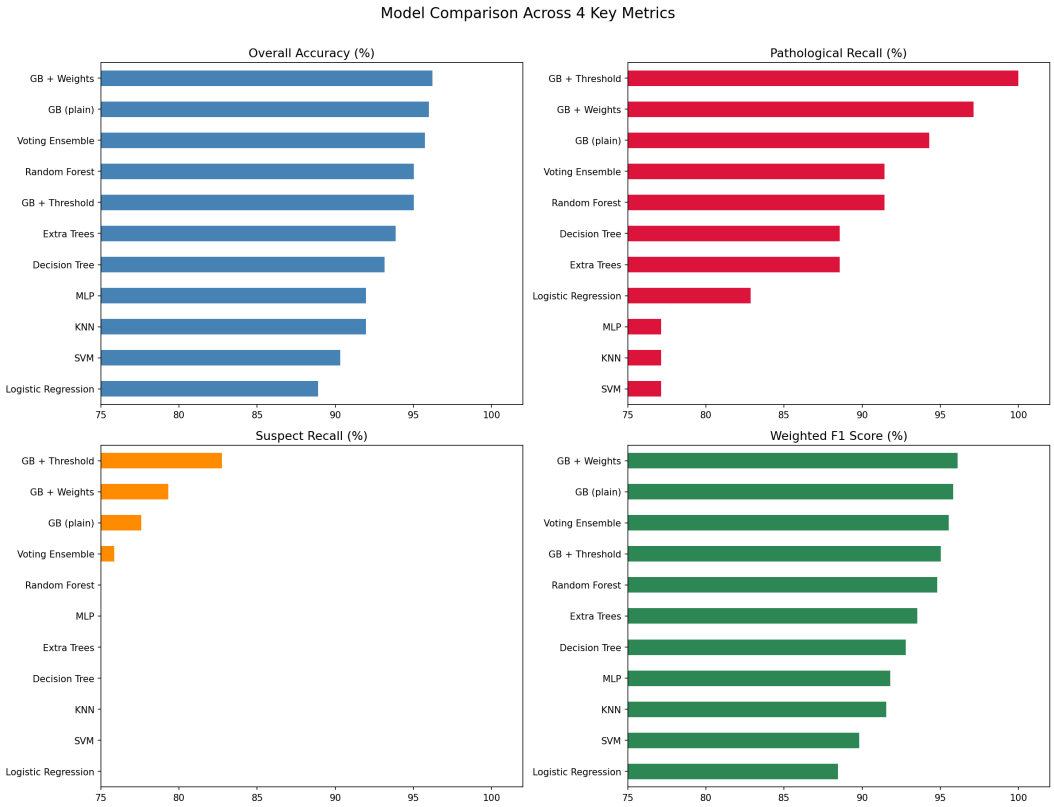


*Fig. 7: Four-Metric Comparison of All Models — Gradient Boosting variants lead on every axis*

## 5.4 Threshold Tuning Results

### Table IV: Probability Threshold Tuning on Gradient Boosting

| Threshold Config | Acc. | Rec.(P) | Rec.(S) | Prec.(P) |
|---|---|---|---|---|
| Default (argmax) | 95.98% | 94.29% | 77.59% | 97.14% |
| P(P)≥0.25, P(S)≥0.3 | 96.22% | 97.14% | 81.03% | — |
| P(P)≥0.10, P(S)≥0.3 | 96.22% | 100.00% | 81.03% | 92.11% |
| P(P)≥0.05, P(S)≥0.2 | 94.56% | 100.00% | 81.03% | 81.40% |

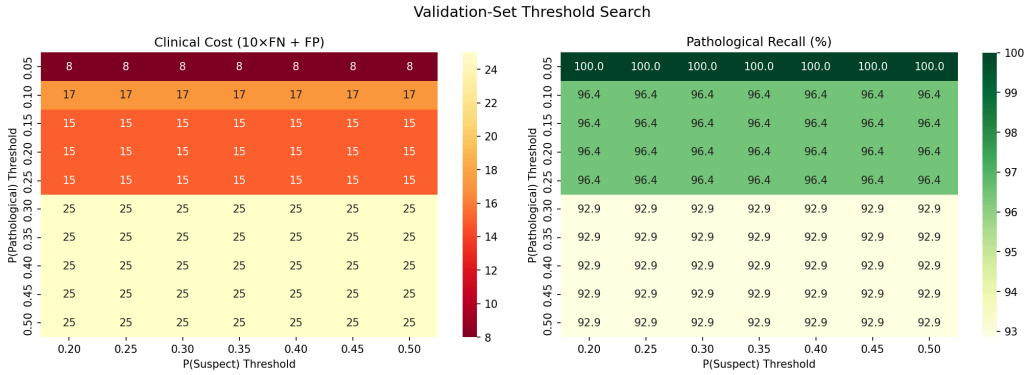


*Fig. 8: Validation-Set Threshold Search — Left: Clinical Cost (10×FN + FP). Right: Pathological Recall*

Lowering the Pathological threshold from default 0.50 to 0.10 captures all 35 Pathological test cases (100% recall) while keeping accuracy at 96.22% and Suspect recall at 81.03%.

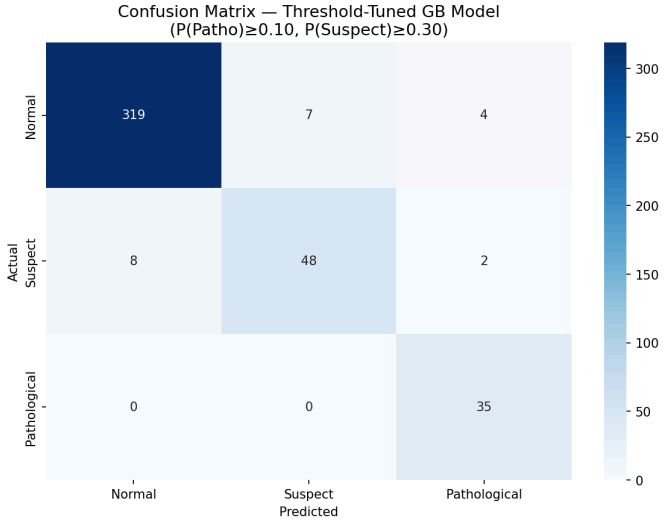


*Fig. 9: Confusion Matrix for the Threshold-Tuned GB Model — All 35 Pathological cases land on the diagonal*

## 5.5 Precision-Recall Trade-off

**Table V: Per-Class Precision and Recall — Default vs. Threshold-Tuned**

| Class | Default Prec. | Default Rec. | Tuned Prec. | Tuned Rec. |
|---|---|---|---|---|
| Normal | 96.47% | 99.39% | 97.01% | 98.48% |
| Suspect | 91.84% | 77.59% | 94.00% | 81.03% |
| Pathological | 97.06% | 94.29% | 92.11% | 100.00% |

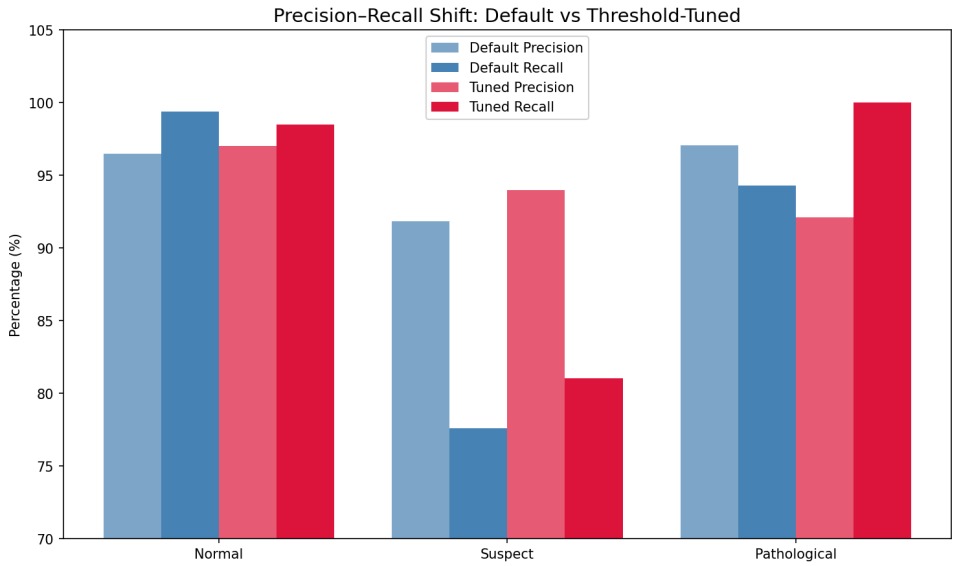


*Fig. 10: Precision-Recall Shift from Default to Threshold-Tuned Classification*

The threshold-tuned model produces only 3 false Pathological flags out of 38 positive predictions (2 Normal and 1 Suspect reclassified upward). Pathological precision drops from 97.06% to 92.11% — roughly 1 false alarm in every 13 alerts. In obstetric practice, this is well within acceptable bounds.

## 5.6 Cross-Validation Results

**Table VII: Five-Fold Stratified CV — Gradient Boosting with Sample Weights**

1. Mean Accuracy: 94.08% ± 1.17%
2. Mean Pathological Recall: 87.86% ± 1.75%
3. Mean Suspect Recall: 76.17% ± 5.12%

Accuracy and Pathological recall are tightly clustered across folds. Suspect recall shows wider spread (±5.12%), expected since the Suspect category sits on the boundary between Normal and Pathological.

# 6. CONCLUSION AND FUTURE SCOPE

## 6.1 Conclusion

This project demonstrates that metric-driven optimization outperforms blanket preprocessing when building classifiers for safety-critical medical applications. The key conclusions are:

1. Gradient Boosting with cost-aware sample weighting (1:5:10) and post-hoc probability threshold tuning achieves 96.22% accuracy with 100% Pathological recall — zero missed pathological cases.
2. Popular preprocessing steps (outlier capping, log transforms, feature engineering, SMOTE) each degrade the top-performing tree ensembles. Preprocessing should be validated empirically per model family.
3. Sample weighting preserves the original geometry of the feature space while steering the optimizer toward minority-class accuracy, outperforming all SMOTE variants.
4. Threshold tuning provides a deployment knob requiring no retraining — hospitals can adjust sensitivity-specificity trade-off based on clinical tolerance.
5. The sequential error-correction mechanism of Gradient Boosting is well-suited to imbalanced multi-class problems, consistently outperforming bagging-based methods.
6. Five-fold stratified CV confirms stable generalization (94.08% ± 1.17% accuracy, 87.86% ± 1.75% Pathological recall).

## 6.2 Discussion

Why Gradient Boosting Wins: The sequential error-correction mechanism concentrates on residuals of the previous ensemble, giving hard-to-classify minority samples progressively more attention. The ranking GB > RF > ET > DT is consistent with this explanation.

When Preprocessing Hurts: Tree ensembles partition feature space with axis-aligned cuts; they are invariant to monotonic transforms and naturally isolate outliers. Capping or compressing features removes the extremes that distinguish Pathological from Normal traces.

The Suspect Bottleneck: Suspect recall (81.03%) remains the weakest metric. This is inherent to the clinical definition — Suspect traces occupy a grey zone, and even experienced clinicians disagree on classification.

Diminishing Returns from Voting Ensembles: Combining GB, RF, and ET in a soft-voting ensemble lowered accuracy relative to GB alone (95.27% vs. 96.22%). When one base learner is substantially stronger, averaging in weaker predictions dilutes the signal.

## 6.3 Future Scope

1. Deep Learning: Applying CNNs or RNNs directly to raw CTG time-series data, bypassing hand-crafted feature extraction.
2. Temporal Modeling: LSTM or Transformer architectures to capture sequential patterns in FHR traces.
3. Multi-Hospital Validation: Testing on CTG data from multiple hospitals to assess generalization across different equipment and populations.
4. Explainability: Integrating SHAP or LIME for interpretable predictions that build clinician trust.
5. Real-Time Deployment: Continuous CTG monitoring system with automated Pathological alerts.
6. Suspect Class Refinement: Hierarchical classification or ordinal regression for the ambiguous Suspect category.
7. Larger Datasets: Validation on CTU-UHB corpus (552 raw recordings) to confirm scalability.

# REFERENCES

[1] Şahin, H., & Subasi, A. (2015). Classification of the cardiotocogram data for anticipation of fetal risks using machine learning techniques. Applied Soft Computing, 33, 231–238.

[2] Rahmayanti, N., Pradani, H., Pahlawan, M., & Vinarti, R. (2022). Comparison of machine learning algorithms to classify fetal health using cardiotocogram data. Procedia Computer Science, 197, 162–171.

[3] Subasi, A., Kadasa, B., & Kremic, E. (2020). Classification of the cardiotocogram data for anticipation of fetal risks using bagging ensemble classifier. Procedia Computer Science, 168, 34–39.

[4] Bache, K. & Lichman, M. (2013). UCI Machine Learning Repository. University of California, School of Information and Computer Science, Irvine, CA.

[5] Cömert, Z. & Kocamaz, A. (2017). Comparison of machine learning techniques for fetal heart rate classification. Acta Physica Polonica A, 132, 451–454.

[6] Cömert, Z., Şengür, A., Budak, U., & Kocamaz, A.F. (2019). Prediction of intrapartum fetal hypoxia considering feature selection algorithms and machine learning models. Health Information Science and Systems, 7, 1–9.

[7] Sundar, C., Chitradevi, M., & Geetharamani, G. (2012). Classification of cardiotocogram data using neural network based machine learning technique. International Journal of Computer Applications, 47, 1–8.

[8] Batra, A., Chandra, A., & Matoria, V. (2017). Cardiotocography analysis using conjunction of machine learning algorithms. In 2017 International Conference on Machine Vision and Information Technology (CMVIT), IEEE, Singapore, pp. 1–6.

[9] World Health Organization. (2023). Maternal mortality. WHO Fact Sheets. Available: https://www.who.int/news-room/fact-sheets/detail/maternal-mortality

# APPENDICES

## Appendix A: Dataset Features

The 21 predictor features extracted from CTG recordings:

1. baseline value — Baseline Fetal Heart Rate (FHR)
2. accelerations — Number of accelerations per second
3. fetal_movement — Number of fetal movements per second
4. uterine_contractions — Number of uterine contractions per second
5. light_decelerations — Number of light decelerations per second
6. severe_decelerations — Number of severe decelerations per second
7. prolongued_decelerations — Number of prolonged decelerations per second
8. abnormal_short_term_variability — Percentage of time with abnormal STV
9. mean_value_of_short_term_variability — Mean value of STV
10. percentage_of_time_with_abnormal_long_term_variability — % time with abnormal LTV
11. mean_value_of_long_term_variability — Mean value of LTV
12. histogram_width — Width of FHR histogram
13. histogram_min — Minimum of FHR histogram
14. histogram_max — Maximum of FHR histogram
15. histogram_number_of_peaks — Number of peaks in histogram
16. histogram_number_of_zeroes — Number of zeroes in histogram
17. histogram_mode — Mode of FHR histogram
18. histogram_mean — Mean of FHR histogram
19. histogram_median — Median of FHR histogram
20. histogram_variance — Variance of FHR histogram
21. histogram_tendency — Tendency of FHR histogram

## Appendix B: Conference Submission Details

This research was submitted to the 2026 IEEE 5th World Conference on Applied Intelligence and Computing (AIC 2026).

1. Paper ID: 2665
2. Title: Computational Intelligence for Fetal Health Classification

3. Authors: Harshit Kukreja, Deepanshu Sethi, Arsh Sudan, Inder Goswami (Chitkara University)
4. Primary Subject Area: Computational Intelligence
5. Submission Date: April 27, 2026

## Appendix C: Project Repository

The complete source code, dataset, and Jupyter notebook are available in the project repository. The notebook (Fetal_Health_Classification.ipynb) contains well-commented code that can be executed sequentially to reproduce all results reported in this project.